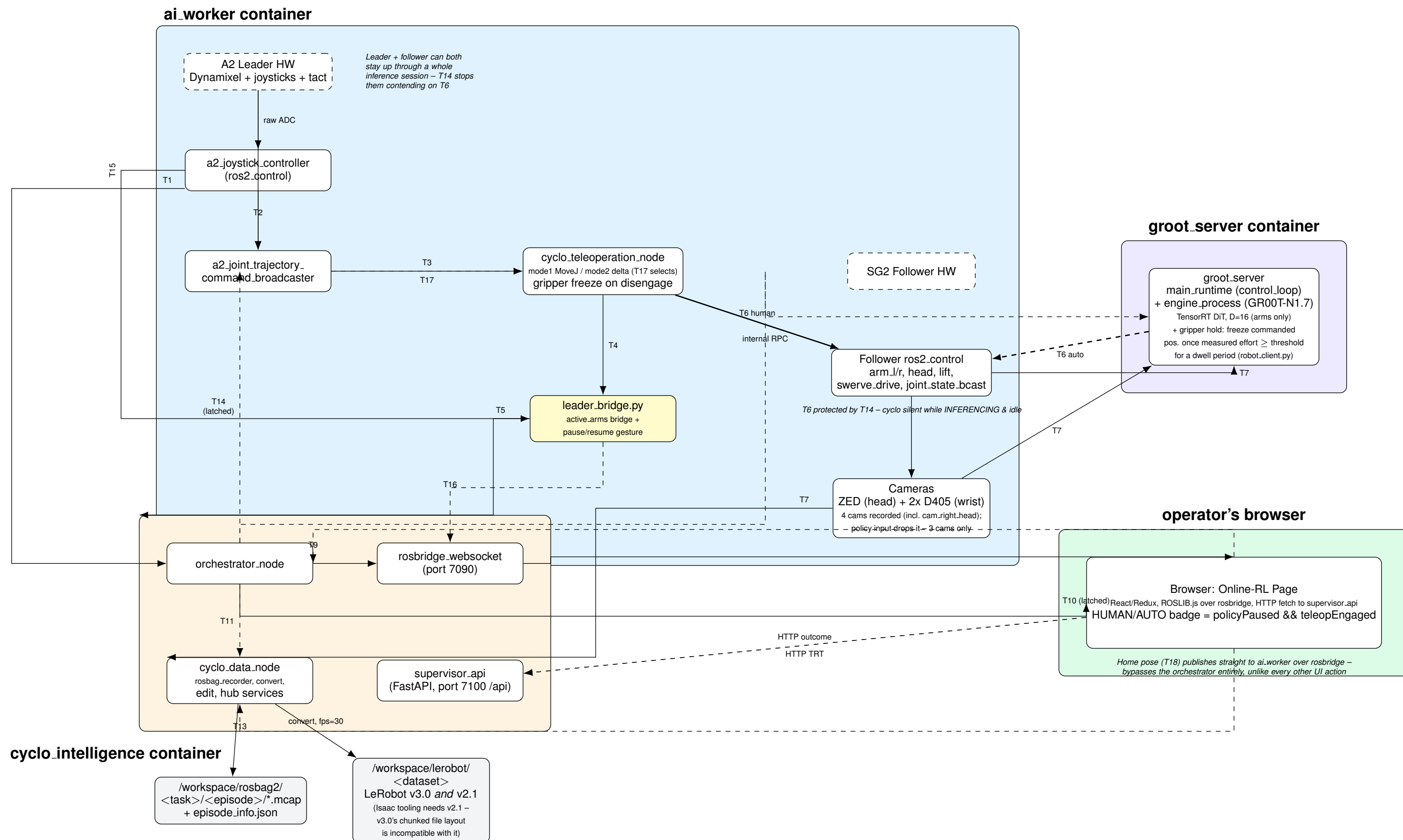


HG-Dagger / Online-RL System Architecture — FFW SG2

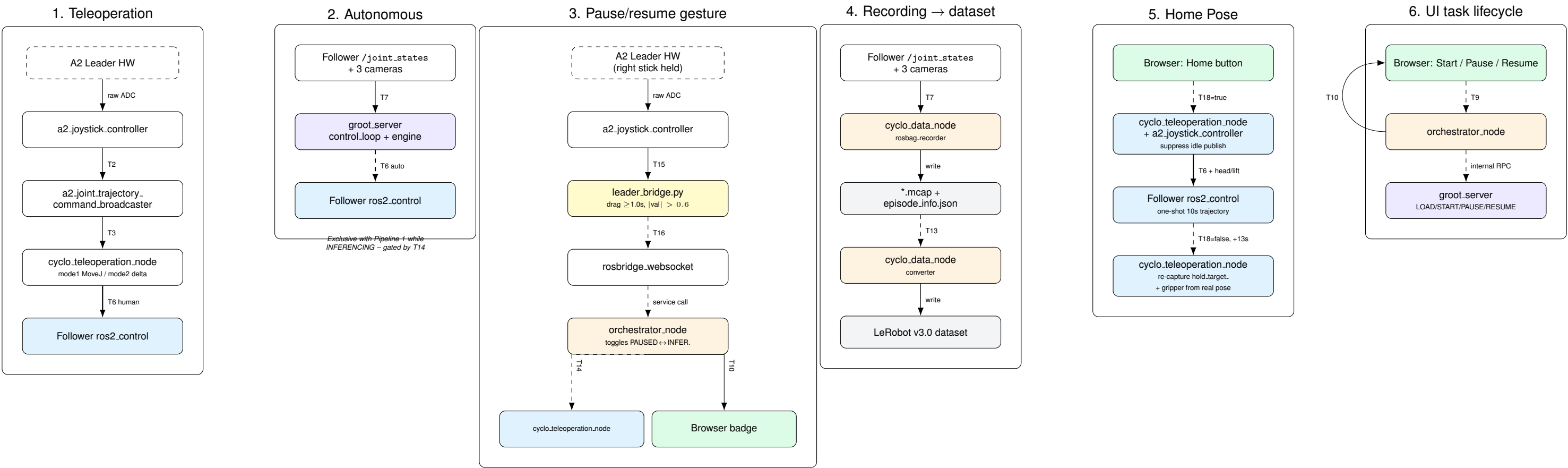
ai_worker (leader + follower hardware) · cyclo_intelligence (orchestration) · groot_server (policy)

Edge labels are topic/service IDs – p.2 walks each core data path step by step, p.3+ defines every I



Core Pipelines — Step by Step

Each box is one node/process; each arrow is one topic or service call, read top to bottom. IDs are defined on p.3+



Topic / Service Legend

ID	Kind	Name	Type	Note
T1	PUB/SUB	.../tact_trigger	std_msgs/String	"left"/"right" tap. Ignored for recording (record_triggers_enabled=false).
T2	PUB/SUB	.../joystick_command	robotis_interfaces /TeleoperationCommand	ENABLE/DISABLE/TOGGLE per arm. a2_joystick_controller → broadcaster.
T3	PUB/SUB	.../raw_joint_trajectory (internal)	trajectory_msgs/JointTrajectory	Broadcaster's leader reference → cyclo_teleoperation_node.
T17	PUB/SUB	.../control_command	robotis_interfaces/ControlModeCommand	Selects mode (1=absolute, 2=delta, default) + enabled arms.
T4	PUB/SUB	.../control_status	robotis_interfaces/ControlModeStatus	active_arms/mode/sync. Not decodable in cyclo_intelligence – excluded from recording.
T5	PUB/SUB latched	.../active_arms_str	std_msgs/String	Decodable translation of T4. UI badge subscribes here.
T15	PUB/SUB	.../sensorxel_joy_values	std_msgs/Float64MultiArray	Raw joystick axes. leader_bridge.py watches R-stick Y for the pause gesture.
T16	PUB/SUB	.../toggle_pause_request	std_msgs/Empty	Fires once on a ≥1.0s drag-hold. Orchestrator decides Pause vs Resume.
T6	PUB/SUB	.../joint_trajectory_command_broadcaster_{l,r}/...	trajectory_msgs/JointTrajectory	Arm commands. 3 possible publishers (teleop/policy/Home); guarded by T14.
T18	PUB/SUB latched	.../home_in_progress	std_msgs/Bool	Bypasses orchestrator. One-shot Home trajectory on T6; falling edge re-captures gripper.
T14	PUB/SUB latched	/task/policy_active	std_msgs/Bool	Mirrors T10's INFERENCING for ai_worker. Gates T6's idle publish.
	PUB/SUB	.../joystick_controller_{l,r}/joint_trajectory	trajectory_msgs/JointTrajectory	Head/lift jog. Frozen during policy control. Left (head) side is republished at 100Hz regardless of jog scale – with the follower's head.controller remapped onto this same topic name, that stream permanently override any other head command (incl. the auto-home-on-startup). Left's publish target is now redirected to a dead-end topic (never reaches the real head.controller); right (lift) is unaffected and still works.
T7	PUB/SUB	/cmd_vel	geometry_msgs/Twist	Mobile base. Frozen during policy control.
	PUB/SUB	/joint_states + camera topics	sensor_msgs/JointState, CompressedImage	Follower feedback + 4 cameras (recorder); policy input uses 3. Feeds broadcaster/policy/recorder/preview. video_recorder.py's subscription to these topics was found commented out with a stray dead-code return in its callback (local, uncommitted edits, not a design issue) – every camera recorded zero frames for every episode until fixed; the raw topics themselves were streaming fine the whole time, making this easy to misdiagnose as a camera/hardware problem.
T9	SERVICE	/task/command	SendCommand.srv	Start/Pause/Resume/Clear policy; Start/Stop/Cancel recording.
T10	PUB/SUB latched	/task/inference_status	interfaces/InferenceStatus	READY/LOADING/INFERENCING/PAUSED phase.
T11	SERVICE	/data/recording	RecordingCommand.srv	Recording lifecycle: orchestrator → cyclo.data.
T13	SERVICE	/data/convert(/status)	srv	LeRobot conversion (v3.0 and/or v2.1). Triggered separately from Save. fps defaults to 30 (was silently 15 – fixed).
T19	SERVICE	/task/command (UPDATE_INSTRUCTION)	SendCommand.srv	Fired by the UI's "Update Task Instruction" button (not the preset dropdown alone). Re-conditions the running policy <i>and</i> appends a (elapsed_s, instruction) entry to the active recording's instruction_change_log via T11/SET_TASK_INFO – this is what lets one continuous multi-phase take end up with correctly time-split segments in episode_info.json instead of one instruction covering the whole episode.
	HTTP	/api/recording/outcome	REST (supervisor_api)	Writes outcome+success_frame to episode_info.json.
	HTTP	/api/backends/groot/trt/*	REST (supervisor_api)	TensorRT engine status/build.

Mode 2 (AiWorkerElbowUpLeaderMode, see T17): anchor-capture on engage, goal = follower_anchor + (leader_now -- leader_anchor) – zero jump even if leader and follower drifted apart. Mode 1 relays absolute leader positions and does jump.

Per-frame dataset columns (converter output)

- intervention (int64) – Causal fill of T10: INFERENCING→0, PAUSED→1, else −1 (also −1 during leader retargeting – that's the clutch, not a human command). Only the −1/0 half is reliable today; the 1 case needs T4→T5 rewiring (pending).
- sample_weight (float32) – Matches Larchenko's LeHome Challenge weighting exactly (arXiv:2606.27163 §9.10, Fig. 17): human frames **2.0**, autonomous baseline **0.3**, ramping to **0.0** over the 5s before a takeover. Excluded frames stay at 1.0.
- reward (int64) – 1 from success_frame onward on outcome=SUCCESS episodes, else 0.
- done (bool) – True on the final frame of the episode only.
- task_index / subtask_index (int64) – Mapped per-frame from episode_info.json's segments array (each with its own frame_duration range and sub_task_instruction) via existing converter logic (_assign_subtask_indices). For a continuous online-RL take this previously always produced exactly one segment spanning the whole episode – every frame got whichever instruction was set last, regardless of what was actually happening for most of the take. instruction_change_log (see T19) now drives splitting that single segment into one correctly-timed sub-segment per real instruction change. The episode's top-level task_instruction (the overall task, e.g. "Screw the orange bolt into the hole using the driver.") is a separate, stable field – it is not derived from the live-changing per-phase instruction, since that field gets overwritten on every T19 click.